- update_all
- update_attribute
- update_column
- update_columns
- update_counters

Note that save also has the ability to skip validations if passed validate: false as argument. This technique should be used with caution.

- save(validate: false)

1.4 valid? and invalid?

To verify whether or not an object is valid, Rails uses the valid? method. You can also use this method on your own. valid? triggers your validations and returns true if no errors were found in the object, and false otherwise. As you saw above:

```
class Person < ActiveRecord::Base
  validates :name, presence: true
end

Person.create(name: "John Doe").valid? # => true
Person.create(name: nil).valid? # => false
```

After Active Record has performed validations, any errors found can be accessed through the errors.messages instance method, which returns a collection of errors. By definition, an object is valid if this collection is empty after running validations.

Note that an object instantiated with new will not report errors even if it's technically invalid, because validations are not run when using new.

```
class Person < ActiveRecord::Base
  validates :name, presence: true
end

>> p = Person.new
# => #<Person id: nil, name: nil>
>> p.errors.messages
# => {}

>> p.valid?
# => false
>> p.errors.messages
# => {name:["can't be blank"]}

>> p = Person.create
# => #<Person id: nil, name: nil>
>> p.errors.messages
# => {name:["can't be blank"]}

>> p.save
# => false

>> p.save!
# => ActiveRecord::RecordInvalid: Validation failed: Name can't be
blank

>> Person.create!
# => ActiveRecord::RecordInvalid: Validation failed: Name can't be
blank
```

invalid? is simply the inverse of valid?. It triggers your validations, returning true if any errors were found in the object, and false otherwise.